

NGINX Reference Guide

Architecture · Configuration · Diagrams · Production Patterns

8

Chapters

25+

Diagrams

50+

Config Snippets

300+

Lines Code

01 Architecture

02 Config

03 Static Hosting

04 Auth

05 Reverse Proxy

06 Load Balancing

07 TLS/SSL

08 Advanced

00 Table of Contents

All 8 chapters at a glance

01 Introduction & Architecture

What is NGINX? · C10K Problem · Event-Driven Model · Installation

02 Configuration Architecture

Context Tree · Location Matching · Variable Reference

03 Hosting Websites & SPAs

Static File Serving · React/Vue SPA Routing · Asset Caching

04 Basic Authentication

htpasswd Setup · Protecting Routes · Security Notes

05 Reverse Proxy Deep Dive

Proxy Flow · Header Forwarding · Timeout Tuning · Caching

06 Load Balancing

Upstream Pools · Four Algorithms · Health Checks & Failover

07 TLS / SSL & HTTPS

TLS Handshake · Certbot & Let's Encrypt · Security Headers

08 Advanced Topics

WebSocket Upgrade · Gzip Pipeline · Rate Limiting · Logging

01

Introduction & Architecture

What NGINX is, why it exists, and how it works internally

What Is NGINX?

NGINX (pronounced **engine-x**) is a high-performance, open-source **web server**, **reverse proxy**, **load balancer**, **HTTP cache**, and **mail proxy**. First released in **2004** by Igor Sysoev, it was designed to overcome Apache's fundamental limitation: handling tens of thousands of simultaneous connections efficiently. Today NGINX powers approximately **34%** of the top 1 million websites worldwide.

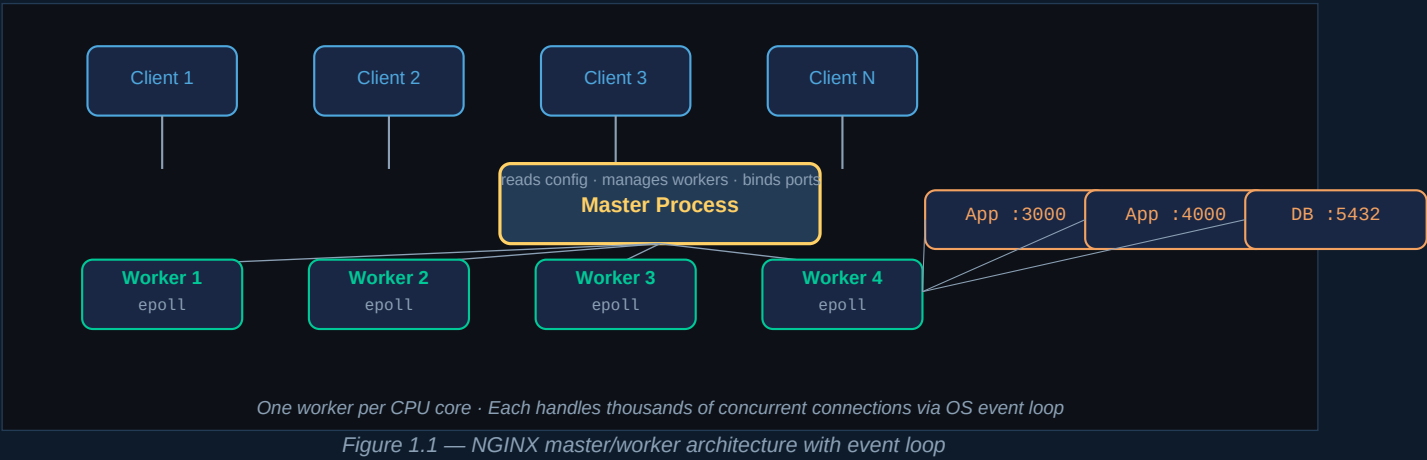
The C10K Problem

In the late 1990s, engineer Dan Kegel posed the challenge: **how do you handle 10,000 concurrent client connections?** Apache's approach — spawning a thread per connection — meant each connection consumed **~2 MB** of RAM. At 10,000 connections: **20 GB** just for idle threads.

Model	Memory (10K conns)	Event Mechanism
Apache (1 thread/conn)	~20 GB RAM	OS thread scheduler
NGINX (event-driven)	~150 MB RAM	epoll (Linux) / kqueue (BSD)

Event-Driven Architecture

NGINX uses a **master/worker process model**. The master reads config and manages workers. Each worker runs an **event loop** that handles thousands of connections simultaneously — no blocking, no thread-per-connection overhead. The OS event system (**epoll** on Linux) notifies the worker exactly when I/O is ready.



Key Architecture Components

Component	Role	Detail
Master Process	Orchestrator	Reads config, manages workers, binds privileged ports
Worker Process	Request Handler	One per CPU core; runs event loop; handles 1000s of conns
Cache Manager	Cache Lifecycle	Removes stale cache entries in background
Cache Loader	Cache Warmup	Loads cache metadata into memory at startup
epoll / kqueue	Event Notification	OS-level API; tells NGINX which FDs are ready

Installation & First Steps

```
# Install NGINX on Ubuntu 22.04 / 24.04
sudo apt update && sudo apt install nginx -y

# Enable service & start
sudo systemctl enable nginx
sudo systemctl start nginx

# Open firewall
sudo ufw allow 'Nginx Full' # port 80 + 443

# Essential workflow
sudo nginx -t # ALWAYS test syntax first
sudo systemctl reload nginx # zero-downtime reload
sudo systemctl restart nginx # full restart (brief downtime)

# Logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

■ Always run `sudo nginx -t` before reloading. A bad config will crash NGINX on restart!

Path	Purpose
/etc/nginx/nginx.conf	Main configuration file — global settings
/etc/nginx/sites-available/	Per-site config files (inactive until symlinked)
/etc/nginx/sites-enabled/	Symlinks to active configs — NGINX reads these
/var/log/nginx/access.log	Every HTTP request logged here
/var/log/nginx/error.log	Errors, warnings, and debug output

02 Configuration Architecture

Context tree, directives, location matching & variables

The Context Tree

NGINX config is a **hierarchical tree of contexts** (blocks) and directives (key-value statements). Outer context directives are **inherited** by inner contexts. The tree: **main** → **events** → **http** → **server** → **location**.

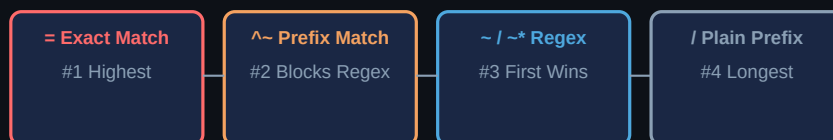
```
# main context (global settings)
user www-data;
worker_processes auto; # auto = one per CPU core
worker_rlimit_nofile 65535;

events {
    worker_connections 1024; # per worker
    use epoll; # Linux kernel event mechanism
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    sendfile on;
    keepalive_timeout 65;
    gzip on;
    server {
        listen 80;
        server_name example.com;
        location / {
            root /var/www/html;
            index index.html;
        }
    }
}
```

Location Block Matching Priority

When NGINX receives a request it follows a **strict priority order** to find the matching location block. Getting this wrong causes subtle routing bugs.



Exact match always wins. ^~ blocks regex from checking. Regex: first match wins. Plain prefix: longest wins.

Figure 2.1 — Location block matching priority

Modifier	Syntax	Priority	Behaviour
Exact	<code>= /exact</code>	1 (highest)	Matches URI exactly. Stops search immediately.
Prefix (no-regex)	<code>^~ /prefix</code>	2	Matches prefix. Blocks regex from checking.
Regex (case-sensitive)	<code>~ /regex</code>	3	First matching regex wins.
Regex (case-insensitive)	<code>~* /regex</code>	3	Same as above, ignores case.
Plain prefix	<code>/prefix</code>	4 (lowest)	Longest matching prefix wins.

```

server {
# Priority 1: exact – only matches GET /
location = / { return 200 'root exact'; }

# Priority 2: prefix – blocks regex for /images/
location ^~ /images/ { root /var/www; }

# Priority 3: regex – matches *.jpg, *.png anywhere
location ~* \.(jpg|png|gif|svg)$ {
expires 1y;
add_header Cache-Control 'public, immutable';
}

# Priority 4: plain prefix fallback
location /api/ { proxy_pass http://localhost:3000/; }

# Catch-all
location / { try_files $uri $uri/ =404; }
}

```

Key Built-in Variables

Variable	Value	Common Use
<code>\$host</code>	Request Host header	Set proxy Host header
<code>\$remote_addr</code>	Client IP address	X-Real-IP header, rate limiting
<code>\$uri</code>	Normalised request URI	<code>try_files</code> , rewrites
<code>\$scheme</code>	http or https	X-Forwarded-Proto, redirects
<code>\$http_upgrade</code>	Upgrade request header	WebSocket detection
<code>\$binary_remote_addr</code>	Client IP (binary)	Rate limit zones (smaller key)
<code>\$request_time</code>	Total request time (s)	Performance logging

<code>\$upstream_response_time</code>	Backend response time	Debugging slow backends
---------------------------------------	-----------------------	-------------------------

03

Hosting Websites & SPAs

Static files, React/Vue apps, caching, custom error pages

Static Website Configuration

`try_files` checks: 1) exact file path, 2) directory index, 3) fallback (e.g. `=404`). The `index` directive specifies fallback filenames when a directory is requested.

```
server {
    listen 80;
    listen [::]:80; # IPv6
    server_name mysite.com www.mysite.com;
    root /var/www/mysite;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    # Aggressive cache for CSS/JS/fonts (1 year)
    location ~* \.(css|js|woff2|woff|ttf)$ {
        expires 1y;
        add_header Cache-Control 'public, immutable';
        access_log off;
    }

    # Custom error pages
    error_page 404 /404.html;
    error_page 500 502 503 504 /50x.html;
}
```

Single-Page Applications (React, Vue, Angular)

SPAs use **client-side routing** — if a user loads `/dashboard/settings` directly, NGINX finds no physical file and returns `404`. The fix: return `index.html` for any path that doesn't match a real file, letting the JS router take over.

```
server {
    listen 80;
    server_name myapp.com;
    root /var/www/myapp/dist; # Vite / CRA build output
    index index.html;

    location / {
        # SPA magic line:
        # 1. Try file 2. Try directory 3. Fall back to index.html
        try_files $uri $uri/ /index.html;
    }

    # API proxy — must NOT fall through to SPA
    location /api/ {
        proxy_pass http://localhost:3000/;
    }
}
```



```
}  
# Cache hashed build artifacts forever  
location ~* \.(js|css)$ {  
    expires 1y;  
    add_header Cache-Control 'public, immutable';  
}  
}
```

■ Without the SPA fallback: refreshing any deep route returns 404. The JS router never even sees the request.

04 Basic Authentication

htpasswd setup, protecting routes, security notes

How HTTP Basic Auth Works

The server returns **401 Unauthorized** with a **WWW-Authenticate: Basic** header. The browser shows a login dialog. Credentials are **Base64-encoded** (NOT encrypted) and sent on every request. **Always use over HTTPS** — Base64 is trivially decoded.

```
# Step 1: Install htpasswd tool
sudo apt install apache2-utils -y

# Step 2: Create password file (-c = create new)
sudo htpasswd -c /etc/nginx/.htpasswd admin

# Add more users (omit -c to append)
sudo htpasswd /etc/nginx/.htpasswd alice

# Step 3: Set correct permissions
sudo chmod 640 /etc/nginx/.htpasswd
sudo chown root:www-data /etc/nginx/.htpasswd
```

Protecting Routes

```
# Protect entire site
server {
    listen 443 ssl;
    auth_basic 'Restricted Area';
    auth_basic_user_file /etc/nginx/.htpasswd;
    location / { try_files $uri $uri/ =404; }
}

# Protect only /admin/ – rest is public
server {
    location / { root /var/www/mysite; }
    location /admin/ {
        auth_basic 'Admin Panel';
        auth_basic_user_file /etc/nginx/.htpasswd;
        proxy_pass http://localhost:3000/admin/;
    }
}

# Sub-path explicitly public (overrides parent auth)
location /admin/public-status {
    auth_basic off;
    proxy_pass http://localhost:3000/status;
}
}
```

■ Basic Auth credentials are Base64-encoded — NOT encrypted. ALWAYS pair with HTTPS/TLS. Without TLS, credentials are visible in plain text on the network.

05 Reverse Proxy Deep Dive

Forwarding requests, header management, caching

What Is a Reverse Proxy?

A reverse proxy sits in front of backend servers and **forwards client requests to them**. From the client's view, they're talking to NGINX only — no knowledge of the backend topology. Benefits: **TLS termination**, **security isolation**, **caching**, **compression**, and **load balancing**.

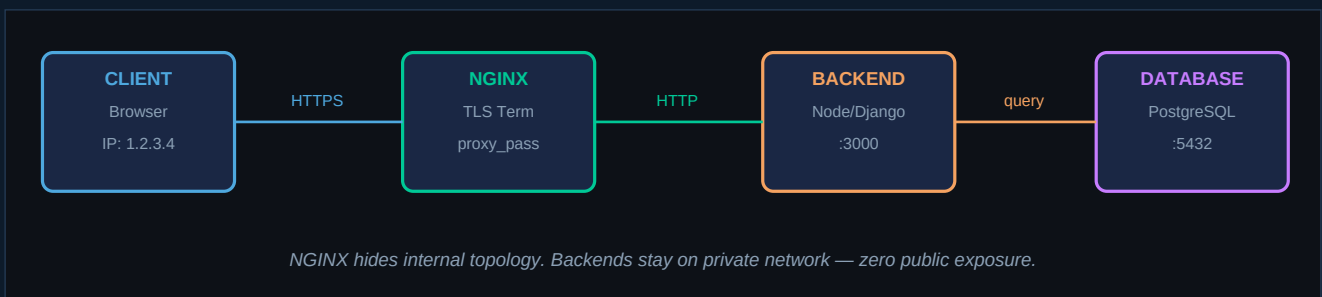


Figure 5.1 — Reverse proxy flow with TLS termination

Basic Reverse Proxy Configuration

```
server {
listen 80;
server_name api.example.com;

location / {
proxy_pass http://localhost:3000;
# NOTE: trailing slash on proxy_pass strips location prefix
# proxy_pass http://localhost:3000; → passes /api/v1 as-is
# proxy_pass http://localhost:3000/; → strips /api/ prefix
# Forward essential headers
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;

# Keep-alive to backend
proxy_http_version 1.1;
proxy_set_header Connection '';

# Timeouts
proxy_connect_timeout 10s;
proxy_send_timeout 60s;
proxy_read_timeout 60s;
}
}
```

Multi-Service Proxy (API Gateway Pattern)

```
server {
```

```
listen 80;
server_name myapp.com;

# Frontend SPA
location / {
    root /var/www/myapp/dist;
    try_files $uri $uri/ /index.html;
}

# REST API - /api/users → http://localhost:4000/users
location /api/ {
    proxy_pass http://localhost:4000/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# ML inference service (slow - extend read timeout)
location /ml/ {
    proxy_pass http://localhost:5001/;
    proxy_read_timeout 120s;
}

# WebSocket service
location /ws/ {
    proxy_pass http://localhost:6000/;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
}
}
```

Proxy Response Caching

```
proxy_cache_path /var/cache/nginx
levels=1:2
keys_zone=my_cache:10m
max_size=1g
inactive=60m
use_temp_path=off;

server {
    location /api/ {
        proxy_pass http://localhost:3000/;
        proxy_cache my_cache;
        proxy_cache_valid 200 10m;
        proxy_cache_valid 404 1m;
        proxy_cache_use_stale error timeout;
        add_header X-Cache-Status $upstream_cache_status;
    }
}
```

06 Load Balancing

Upstream pools, algorithms, health checks & session persistence

How NGINX Load Balancing Works

Load balancing is done through **upstream pools**. You define a named group of servers, then `proxy_pass` to the pool name. NGINX selects which server handles each request based on the **algorithm** you choose. If a server fails (`max_fails` threshold), NGINX stops sending to it until `fail_timeout` elapses.

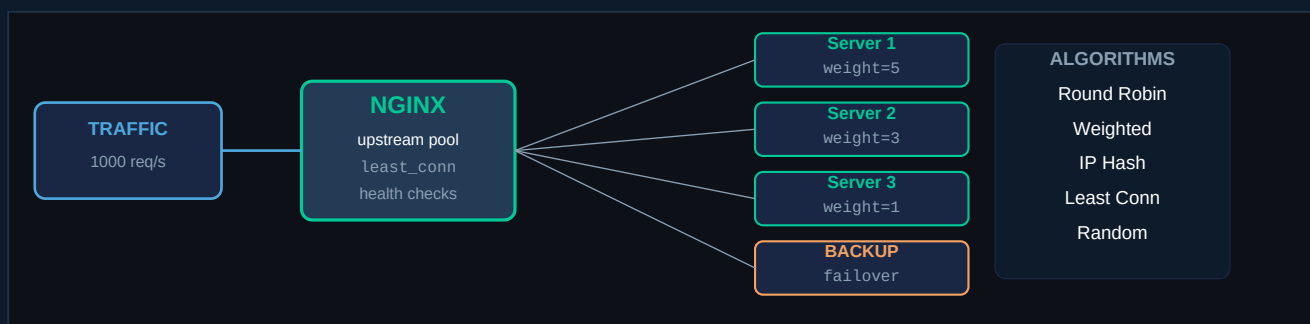


Figure 6.1 — Upstream pool with weighted servers and failover

Upstream Configuration

```

upstream app_servers {
# Algorithm: least connections (best for variable request durations)
least_conn;

# Primary servers with weights
server 192.168.1.10:3000 weight=5 max_fails=3 fail_timeout=30s;
server 192.168.1.11:3000 weight=3 max_fails=3 fail_timeout=30s;
server 192.168.1.12:3000 weight=1 max_fails=3 fail_timeout=30s;

# Backup: only used when ALL primary servers are down
server 192.168.1.20:3000 backup;

# Manually disabled (e.g. during maintenance)
server 192.168.1.99:3000 down;

# Keep persistent connections to backends
keepalive 32;
}

server {
listen 80;
location / {
proxy_pass http://app_servers;
proxy_set_header Host $host;
proxy_http_version 1.1;
proxy_set_header Connection '';
}
}
  
```

The Four Algorithms Explained

Algorithm	Directive	How It Works	Best For
Round Robin	(default)	Sequential distribution across servers	Equal-capacity, uniform request times
Weighted RR	weight=N	Proportional requests by weight	Mixed-capacity servers
IP Hash	ip_hash;	Same client IP → always same server	Stateful apps, session affinity
Least Connections	least_conn;	Send to server with fewest active conns	Variable-length requests, long polling

Health Checks & Passive Failover

NGINX Open Source uses **passive health checks**: if a server returns errors or times out, it's marked unhealthy for `fail_timeout` seconds. After that window, NGINX sends one test request — if it succeeds, the server is restored to the pool.

```
upstream backend {
server 10.0.0.1:3000 max_fails=3 fail_timeout=30s;
# After 3 failures within 30s → removed for 30s
# After 30s → one test request; if success → restored
server 10.0.0.2:3000 max_fails=3 fail_timeout=30s;
server 10.0.0.3:3000 backup;
}
```

07

TLS / SSL & HTTPS

Certificates, HTTPS setup, security headers & TLS handshake

How TLS Works — The Handshake

TLS encrypts all data between client and server. Before HTTP flows, client and server perform a **cryptographic handshake** to authenticate the server and derive shared session keys. **TLS 1.3** (the current standard) does this in just **1 round-trip** vs TLS 1.2's 2.

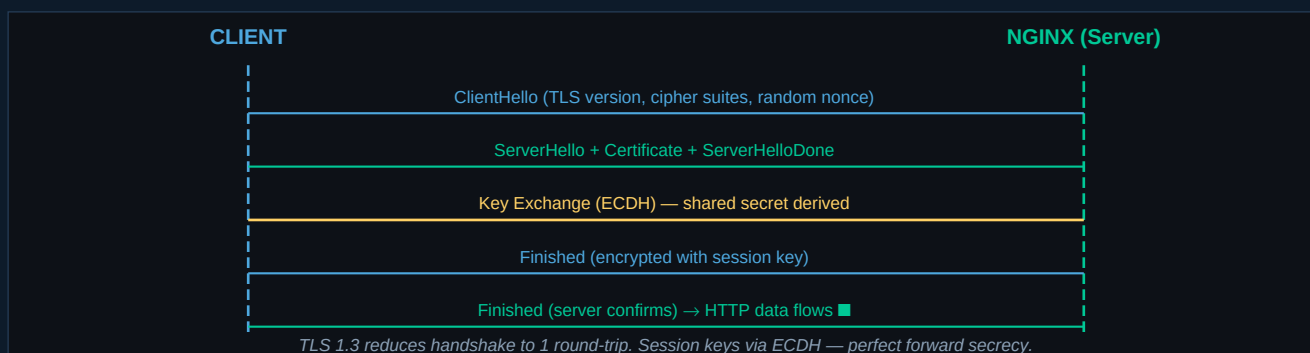


Figure 7.1 — TLS 1.3 handshake sequence

Let's Encrypt & Certbot

Let's Encrypt provides **free**, **automated**, **90-day** TLS certificates. Certbot is the official client that handles issuance and **automatic renewal**. It integrates directly with NGINX to edit your config automatically.

```
# 1. Install Certbot with NGINX plugin
sudo apt install certbot python3-certbot-nginx -y

# 2. Obtain certificate (Certbot edits nginx config automatically)
sudo certbot --nginx -d example.com -d www.example.com

# 3. Dry run to test renewal
sudo certbot renew --dry-run

# 4. Check renewal timer (auto-renews when < 30 days from expiry)
sudo systemctl status certbot.timer

# 5. List all certificates
sudo certbot certificates
```

Full HTTPS Server Configuration

```
# Redirect HTTP → HTTPS
server {
    listen 80; listen [::]:80;
    server_name example.com www.example.com;
    return 301 https://$host$request_uri;
}

# HTTPS server
server {
    listen 443 ssl http2;
```

```
server_name example.com www.example.com;

# Certificate paths (set by certbot)
ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

# Modern TLS settings
ssl_protocols TLSv1.2 TLSv1.3;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off; # perfect forward secrecy

# Security headers
add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains' always;
add_header X-Frame-Options DENY always;
add_header X-Content-Type-Options nosniff always;
add_header Referrer-Policy 'strict-origin-when-cross-origin' always;

location / { try_files $uri $uri/ =404; }
```

Security Headers Explained

Header	Value	Protects Against
Strict-Transport-Security	max-age=31536000; includeSubDomains	Downgrade attacks — forces HTTPS for 1 year
X-Frame-Options	DENY	Clickjacking — prevents loading in
X-Content-Type-Options	nosniff	MIME type sniffing attacks
Referrer-Policy	strict-origin-when-cross-origin	Leaking full URL to third parties
Permissions-Policy	geolocation=(), microphone=()	Unwanted browser feature access
Content-Security-Policy	default-src 'self'	XSS by restricting script sources

08 Advanced Topics

WebSockets, gzip, rate limiting, proxy caching, structured logging

WebSocket Proxying

WebSocket is a **full-duplex communication protocol**. It starts as an HTTP request with an **Upgrade** header. NGINX must forward these upgrade headers and use **HTTP/1.1** — WebSocket is **not compatible** with HTTP/2 at the proxy level.

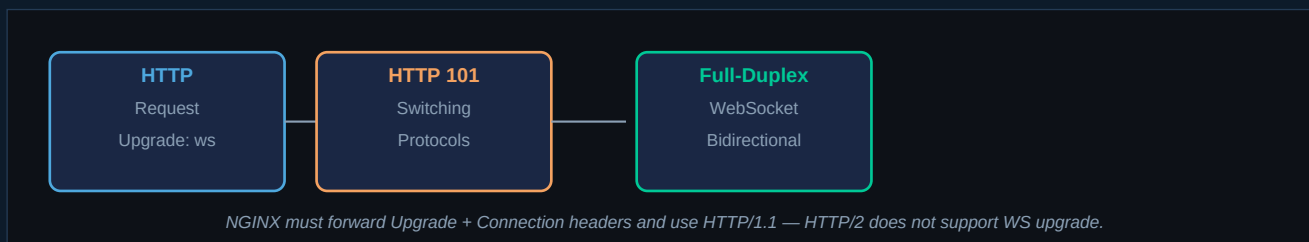


Figure 8.1 — WebSocket upgrade from HTTP to WS

```
# Map upgrade header to connection value
map $http_upgrade $connection_upgrade {
    default upgrade;
    '' close;
}

server {
    listen 443 ssl http2;
    server_name ws.example.com;

    location /ws/ {
        proxy_pass http://localhost:8080/;
        proxy_http_version 1.1;

        # These two headers trigger the upgrade
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;

        # Keep WS connections alive (default 60s is too short)
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}
```

Gzip Compression

Gzip reduces text responses by **~70%**. Only compress text-based content — images/video are already compressed; double-compressing wastes CPU with no benefit.

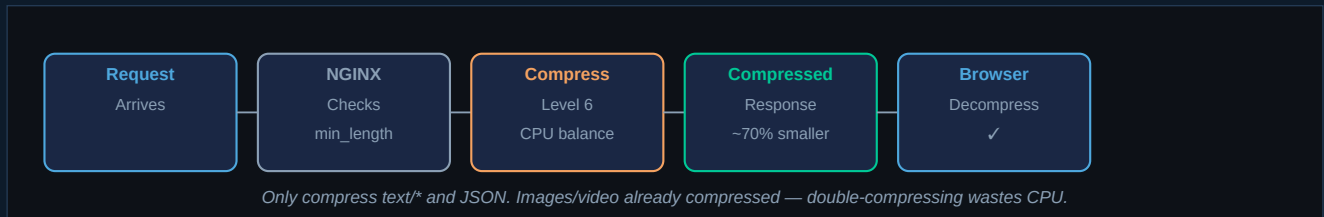


Figure 8.2 — Gzip compression pipeline

```

http {
gzip on;
gzip_comp_level 6; # 1-9: 6 is best CPU/size tradeoff
gzip_min_length 256; # don't compress tiny responses
gzip_proxied any; # compress proxied responses too
gzip_vary on; # add Vary: Accept-Encoding header
# NEVER compress images/video — already compressed!
gzip_types
text/plain text/css text/javascript
application/json application/javascript
application/xml image/svg+xml font/woff2;
}

```

Rate Limiting

NGINX uses the **leaky bucket algorithm**. Requests fill the bucket; bucket drains at the defined rate. `burst` allows short spikes. `nodeLAY` serves bursts immediately rather than queuing them.

```

http {
# 10MB zone stores ~160,000 IPs
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=2r/m;

server {
# API: 10 req/s, burst of 20 served immediately
location /api/ {
limit_req zone=api burst=20 nodeLAY;
limit_req_status 429; # HTTP 429 Too Many Requests
}

# Login: max 2/min (brute-force protection)
location /auth/login {
limit_req zone=login burst=3 nodeLAY;
}
}
}

```

Structured JSON Access Logging

```

http {
# JSON logs — easy to parse with jq, Datadog, Elasticsearch
log_format json_combined escape=json
'{'
'"time":'$time_iso8601','
'"method":'$request_method','

```

```
"uri": "$uri", '
"status": $status, '
"request_time": $request_time, '
"remote_addr": "$remote_addr" '
}';
access_log /var/log/nginx/access.log json_combined;
}
```

✓ JSON logs are parseable by jq, Splunk, Datadog, and Elasticsearch out of the box — prefer them over the default combined log format for production.

Performance Tuning Checklist

Setting	Recommended Value	Why
worker_processes	auto	One per CPU core — auto detects
worker_connections	1024–4096	Max concurrent connections per worker
worker_rlimit_nofile	65535	Raise OS file descriptor limit
sendfile	on	Kernel-level file transfer, avoids user-space copy
tcp_nopush	on	Batch TCP headers with sendfile
keepalive_timeout	65	Keep TCP connections alive 65s
gzip_comp_level	6	Best CPU-vs-size balance

09

Quick-Reference Cheat Sheet

Most-used commands and directives at a glance

Task	Command / Directive
Test config syntax	<code>sudo nginx -t</code>
Reload without downtime	<code>sudo systemctl reload nginx</code>
View error log live	<code>sudo tail -f /var/log/nginx/error.log</code>
HTTP → HTTPS redirect	<code>return 301 https://\$host\$request_uri;</code>
SPA / React fallback	<code>try_files \$uri \$uri/ /index.html;</code>
Basic reverse proxy	<code>proxy_pass http://localhost:PORT;</code>
Forward client IP	<code>proxy_set_header X-Real-IP \$remote_addr;</code>
Load balancer pool	<code>upstream p { least_conn; server IP:PORT; }</code>
Enable basic auth	<code>auth_basic 'Msg'; auth_basic_user_file /path;</code>
WebSocket upgrade	<code>proxy_set_header Upgrade \$http_upgrade;</code>
Rate limit zone	<code>limit_req_zone \$binary_remote_addr zone=z:10m rate=10r/s;</code>
Apply rate limit	<code>limit_req zone=z burst=20 nodelay;</code>
Enable gzip	<code>gzip on; gzip_comp_level 6;</code>
Cache static assets 1yr	<code>expires 1y; add_header Cache-Control 'public, immutable';</code>
HSTS header	<code>add_header Strict-Transport-Security 'max-age=31536000' always;</code>
Custom error page	<code>error_page 404 /404.html;</code>
Install certificate	<code>sudo certbot --nginx -d example.com</code>
Check renewal timer	<code>sudo systemctl status certbot.timer</code>

✓ Debugging tip: Always run `sudo nginx -t` before reloading. Check `/var/log/nginx/error.log` first. Temporarily increase log level with: `error_log /var/log/nginx/error.log debug;`